



Underserved Intercity Air Routes

BUSN 32120 Data Analysis with Python
Final Project Presentation
Beatriz Mandetta, Joyce Guo, Sophie Wu



Project overview & Problem statement

Which major-city air routes appear underserved?

Indicators:



**Flight
frequency**



**City
population**



**Tourism
demand**



**Economic
capacity**

Data sources:



Flight records



OurAirports

Airports geography



City population (WUP)



GDP data

Database included vs. midterm



Target audience & Main goal

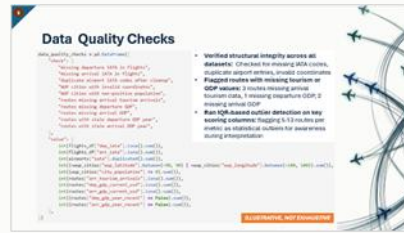


**Airlines route
planners and
aviation analysts**

Enable airline planners to quickly identify **high-potential city pairs** where **route demand outpaces current flight supply**, guiding deeper commercial research and **network expansion opportunities**



Project Timeline & Workflow



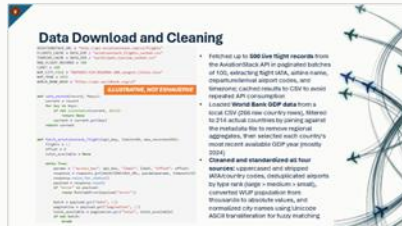
Import and setup
(e.g., Pandas, Numpy, Scikit-learn, Matplotlib, Seaborn, SQLite)

Data quality checks
(missing values, duplicates, plausibility)

Feature engineering
with demand scores
(population, tourism, GDP)

Modeling (e.g. linear and logistic regression to test explanatory power of scores)

Data download and cleaning (flights, airports, population, tourism, GDP)



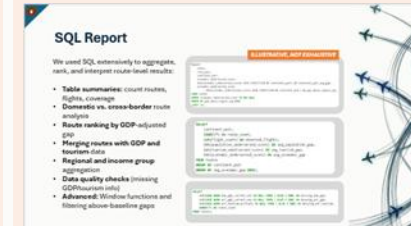
Merging datasets:
enriching routes with demand features



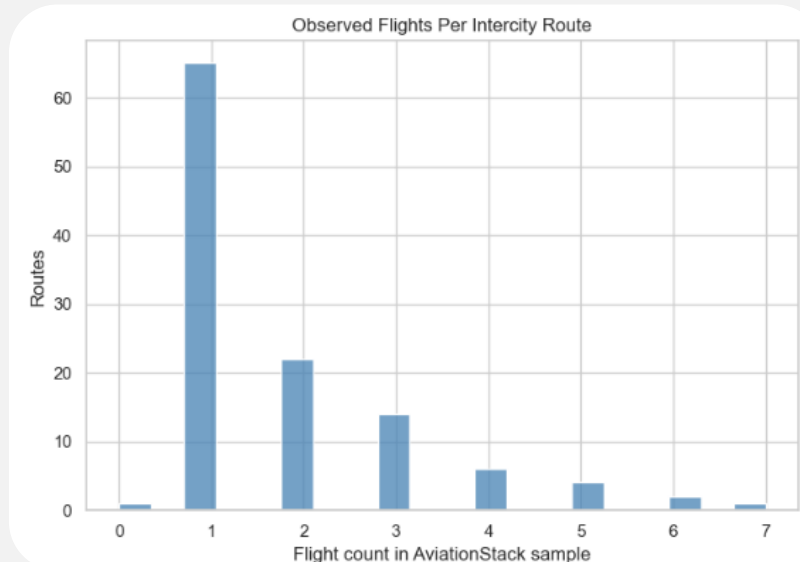
EDA: Statistical summaries and visualizations



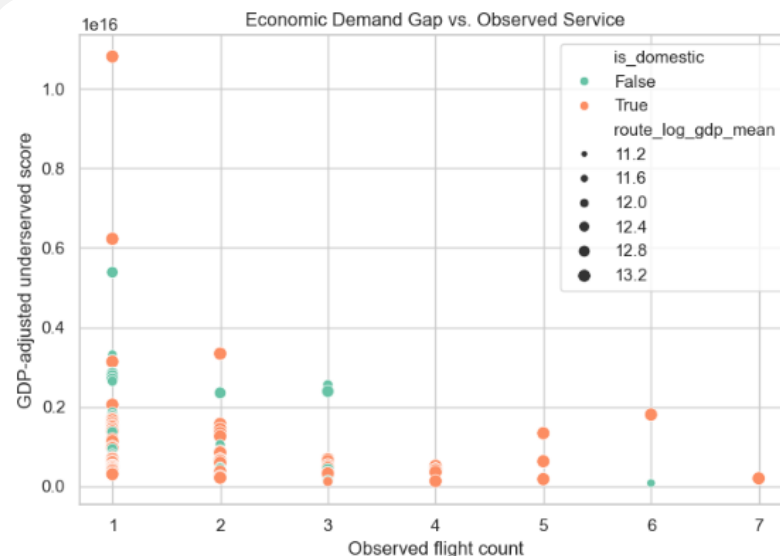
Exporting outputs for SQL and reporting



EDA & Main insights



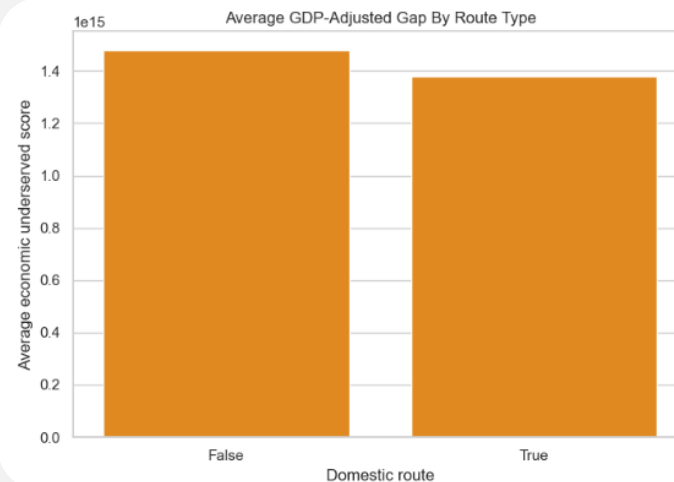
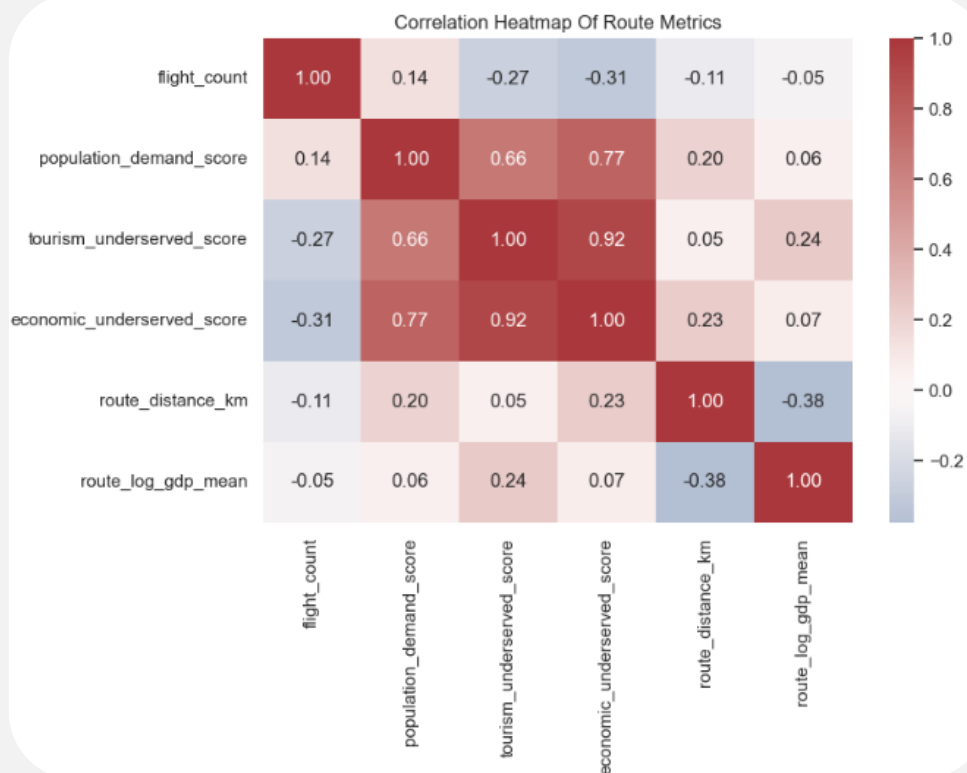
- Over 80 of 115 routes appear only once in the sample - supply is **extremely sparse** across most city pairs
- The **right-skewed distribution** confirms this dataset captures **underserved corridors**, not established high-frequency routes



- The highest gap scores all cluster at flight count = 1, showing **low supply** (not low demand) **drives the underserved ranking**
- **Domestic routes** (orange) **dominate the top gap scores**; larger bubbles (higher GDP) at the top confirm **wealthy city pairs are the most underserved**

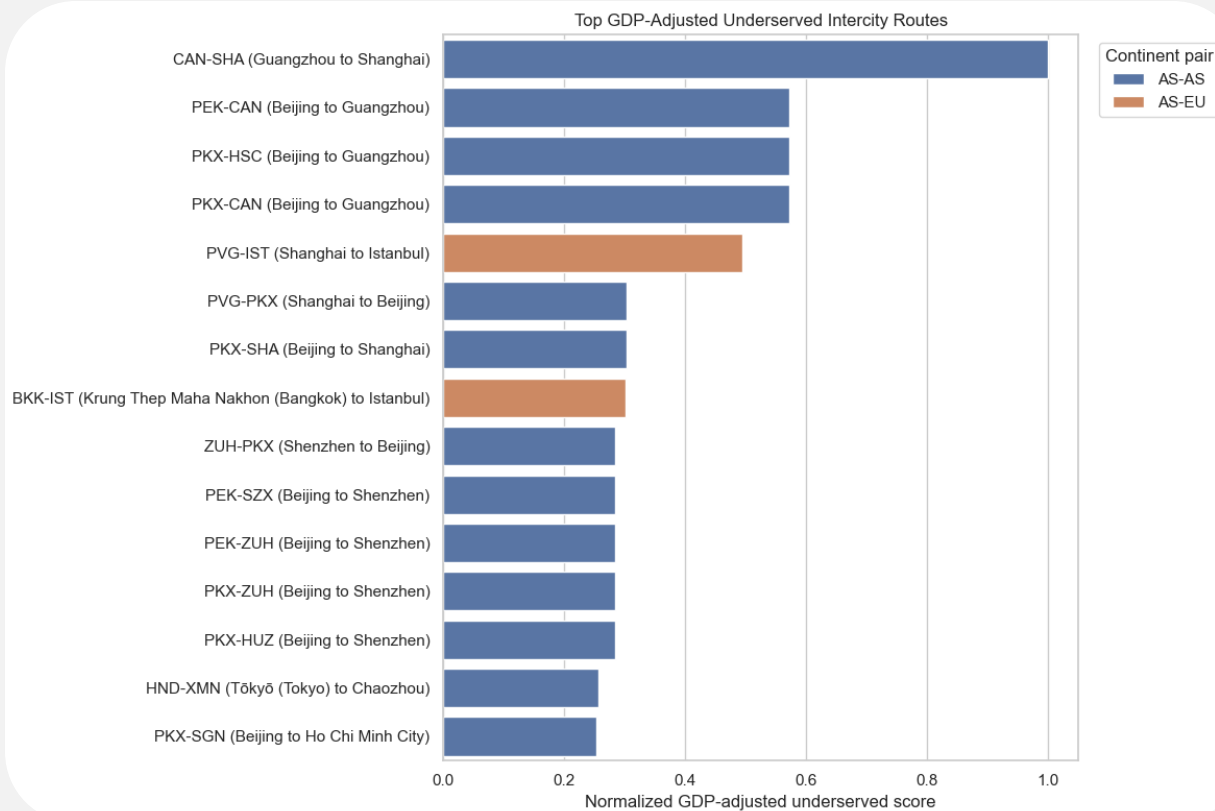


EDA & Main insights



- economic_underserved_score & tourism_underserved_score are nearly identical ($r = 0.95$), meaning **both metrics flag the same routes**
- population_demand_score drives the ranking most strongly ($r = 0.77$), meaning **city size is the primary underserved signal**
- Domestic and cross-border routes have nearly equal average gap scores

EDA & Main insights



- Top 15 routes are almost entirely **intra-Asian (AS-AS)**, dominated by Chinese city pairs (Beijing, Shanghai, Guangzhou, Shenzhen)
- **CAN-SHA (Guangzhou → Shanghai)** is a clear outlier at score = 1.0, combining two of the world's largest metro populations with only 1 observed flight
- The only cross-continental entries (PVG-IST and BKK-IST) flag Asia-Europe as a secondary underserved corridor



Model 1: linear regression for observed route service

ILLUSTRATIVE, NOT EXHAUSTIVE

```
# Model 1: linear regression for observed route service.
X = model_df[model_features]
y = np.log1p(model_df["flight_count"])
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)
linear_model = Pipeline([("preprocess", preprocess), ("model", LinearRegression())])
linear_model.fit(X_train, y_train)
y_pred = linear_model.predict(X_test)
linear_metrics = pd.DataFrame({
    "metric": ["R-squared", "RMSE", "MAE", "test rows"],
    "value": [
        r2_score(y_test, y_pred),
        np.sqrt(mean_squared_error(y_test, y_pred)),
        mean_absolute_error(y_test, y_pred),
        len(y_test),
    ],
})
```

- **Target:** $\log_{1p}(\text{flight_count})$ - how much service does a route actually get?
- **Result:** $R^2 = -0.004$ · $\text{RMSE} = 0.37$ · $\text{MAE} = 0.31$
- **Takeaway:** Features cannot predict observed flights - supply in this 500-record sample is effectively random, not demand-driven



Model 2: logistic regression for whether a route is in the top economic-gap quartile

Model 2: logistic regression for whether a route is in the top economic-gap quartile.

```

y_class = model_df["top_economic_gap"].astype(int)
stratify_target = y_class if y_class.value_counts().min() >= 2 else None
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X, y_class, test_size=0.30, random_state=42, stratify=stratify_target
)
logistic_model = Pipeline([("preprocess", preprocess), ("model", LogisticRegression(max_iter=1000, class_weight="balanced"))])
logistic_model.fit(X_train_c, y_train_c)
y_pred_c = logistic_model.predict(X_test_c)
y_proba_c = logistic_model.predict_proba(X_test_c)[:, 1]
roc_auc = roc_auc_score(y_test_c, y_proba_c) if y_test_c.nunique() == 2 else np.nan
logistic_metrics = pd.DataFrame({
    "metric": ["Accuracy", "Precision", "Recall", "F1", "ROC AUC", "test rows"],
    "value": [
        accuracy_score(y_test_c, y_pred_c),
        precision_score(y_test_c, y_pred_c, zero_division=0),
        recall_score(y_test_c, y_pred_c, zero_division=0),
        f1_score(y_test_c, y_pred_c, zero_division=0),
        roc_auc,
        len(y_test_c),
    ],
})

```

ILLUSTRATIVE, NOT EXHAUSTIVE

- **Target:** Binary - is this route in the top 25% economic gap quartile?
- **Result:** Accuracy = 82% · ROC AUC = 0.91 · F1 = 0.57 · Recall = 44%
- **Takeaway:** The model identifies **high-gap routes** well overall (AUC = 0.91), but misses ~half of true positives - a useful screening tool, not a definitive classifier



SQL Report

We used SQL extensively to aggregate, rank, and interpret route-level results:

- **Table summaries:** count routes, flights, coverage
- **Domestic vs. cross-border** route analysis
- **Route ranking by GDP-adjusted gap**
- **Merging routes with GDP and tourism data**
- **Regional and income group aggregation**
- **Data quality checks** (missing GDP/tourism info)
- **Advanced:** Window functions and filtering above-baseline gaps

ILLUSTRATIVE, NOT EXHAUSTIVE

```
SELECT
  route,
  city_pair,
  continent_pair,
  economic_underserved_score,
  AVG(economic_underserved_score) OVER (PARTITION BY continent_pair) AS continent_pair_avg_gap,
  economic_underserved_score
  - AVG(economic_underserved_score) OVER (PARTITION BY continent_pair) AS gap_above_region_avg
FROM routes
WHERE economic_underserved_score IS NOT NULL
ORDER BY gap_above_region_avg DESC
LIMIT 20;
```

```
SELECT
  continent_pair,
  COUNT(*) AS route_count,
  SUM(flight_count) AS observed_flights,
  AVG(population_underserved_score) AS avg_population_gap,
  AVG(tourism_underserved_score) AS avg_tourism_gap,
  AVG(economic_underserved_score) AS avg_economic_gap
FROM routes
GROUP BY continent_pair
ORDER BY avg_economic_gap DESC;
```

```
SELECT
  SUM(CASE WHEN dep_gdp_current_usd IS NULL THEN 1 ELSE 0 END) AS missing_dep_gdp,
  SUM(CASE WHEN arr_gdp_current_usd IS NULL THEN 1 ELSE 0 END) AS missing_arr_gdp,
  SUM(CASE WHEN arr_tourism_arrivals IS NULL THEN 1 ELSE 0 END) AS missing_arr_tourism,
  COUNT(*) AS route_count
FROM routes;
```



Conclusion

Key findings



- **Chinese mega-city corridors** dominate the top underserved rankings, with Guangzhou–Shanghai as the clear outlier
- **AS-EU routes** (Shanghai–Istanbul, Bangkok–Istanbul) lead cross-continental candidates
- Airlines are not routing proportionally to demand - validating the **gap-score framework**

Limitations



- **500-record** API snapshot cannot represent true route frequency
- Population, tourism, and GDP are **proxies**, not actual commercial data
- **Sample too small** for models to generalize

Next steps



- Replace snapshot with **full-year scheduled capacity data**
- Add **route-level constraints** (visa policies, airport slots)
- Swap country-level proxies for **city-level metrics**





Thank you!



APPENDIX



Import and Setup

```
import math
import os
import re
import sqlite3
import unicodedata
import warnings
from getpass import getpass
from pathlib import Path

warnings.filterwarnings("ignore", message="urllib3 v2 only supports OpenSSL.*")

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import requests
import seaborn as sns
from IPython.display import display
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    f1_score,
    mean_absolute_error,
    mean_squared_error,
    precision_score,
    r2_score,
    recall_score,
    roc_auc_score,
)
```

ILLUSTRATIVE, NOT EXHAUSTIVE

- **Imported the full analytical stack:** **Pandas & NumPy** for data manipulation, **Scikit-learn** for modeling pipelines, **Matplotlib & Seaborn** for visualization, **SQLite3** for SQL export, and **Requests** for live API calls
- **Configured global environment:** suppressed irrelevant warnings, set Seaborn whitegrid theme, expanded Pandas column display to 120 columns, and initialized a local data/directory for caching
- **Defined all key constants upfront** - API endpoint, file paths for GDP/WUP/airport sources, max flight record cap (500), and the WUP reference year (2025) - to make the pipeline reproducible across reruns



Data Download and Cleaning

```
AVIATIONSTACK_URL = "http://api.aviationstack.com/v1/flights"
FLIGHTS_CACHE = DATA_DIR / "aviationstack_flights_cached.csv"
TOURISM_CACHE = DATA_DIR / "world_bank_tourism_cached.csv"
MAX_FLIGHT_RECORDS = 500
LIMIT = 100
WUP_CITY_FILE = "WUP2025-F18-DEGURBA-100_Largest_Cities.xlsx"
WUP_YEAR = 2025
WORLD_BANK_BASE = "https://api.worldbank.org/v2"
```

ILLUSTRATIVE, NOT EXHAUSTIVE

```
def safe_nested(record, *keys):
    current = record
    for key in keys:
        if not isinstance(current, dict):
            return None
        current = current.get(key)
    return current

def fetch_aviationstack_flights(api_key, limit=100, max_records=500):
    flights = []
    offset = 0
    total_available = None

    while True:
        params = {"access_key": api_key, "limit": limit, "offset": offset}
        response = requests.get(AVIATIONSTACK_URL, params=params, timeout=30)
        response.raise_for_status()
        payload = response.json()
        if "error" in payload:
            raise RuntimeError(payload["error"])

        batch = payload.get("data", [])
        pagination = payload.get("pagination", {})
        total_available = pagination.get("total", total_available)
        if not batch:
            break
```

- Fetched up to **500 live flight records** from the AviationStack API in paginated batches of 100, extracting flight IATA, airline name, departure/arrival airport codes, and timezone; cached results to CSV to avoid repeated API consumption
- Loaded **World Bank GDP data** from a local CSV (266 raw country rows), filtered to 214 actual countries by joining against the metadata file to remove regional aggregates, then selected each country's most recent available GDP year (mostly 2024)
- **Cleaned and standardized all four sources:** uppercased and stripped IATA/country codes, deduplicated airports by type rank (large > medium > small), converted WUP population from thousands to absolute values, and normalized city names using Unicode ASCII transliteration for fuzzy matching



Data Quality Checks

```
data_quality_checks = pd.DataFrame({
    "check": [
        "missing departure IATA in flights",
        "missing arrival IATA in flights",
        "duplicate airport IATA codes after cleanup",
        "WUP cities with invalid coordinates",
        "WUP cities with non-positive population",
        "routes missing arrival tourism arrivals",
        "routes missing departure GDP",
        "routes missing arrival GDP",
        "routes with stale departure GDP year",
        "routes with stale arrival GDP year",
    ],
    "value": [
        int(flights_df["dep_iata"].isna().sum()),
        int(flights_df["arr_iata"].isna().sum()),
        int(airports["iata"].duplicated().sum()),
        int((~wup_cities["wup_latitude"].between(-90, 90) | ~wup_cities["wup_longitude"].between(-180, 180)).sum()),
        int((wup_cities["city_population"] <= 0).sum()),
        int(routes["arr_tourism_arrivals"].isna().sum()),
        int(routes["dep_gdp_current_usd"].isna().sum()),
        int(routes["arr_gdp_current_usd"].isna().sum()),
        int((routes["dep_gdp_year_recent"] == False).sum()),
        int((routes["arr_gdp_year_recent"] == False).sum()),
    ],
})
```

- **Verified structural integrity across all datasets:** Checked for missing IATA codes, duplicate airport entries, invalid coordinates
- **Flagged routes with missing tourism or GDP values:** 3 routes missing arrival tourism data, 1 missing departure GDP, 2 missing arrival GDP
- **Ran IQR-based outlier detection on key scoring columns:** flagging 5-13 routes per metric as statistical outliers for awareness during interpretation

ILLUSTRATIVE, NOT EXHAUSTIVE



Merging Datasets

```
airport_city_matches = match_airport_to_nearest_wup_city(airports, wup_cities, max_distance_km=250)

# Merge flights to city endpoints, tourism, and GDP.
endpoint_cols = [
    "iata", "airport_name", "airport_city", "airport_country_code", "airport_continent",
    "airport_latitude", "airport_longitude", "country_code", "country_iso3", "country_name",
    "wup_city", "city_population", "city_rank", "airport_to_wup_city_km",
]
dep_endpoints = airport_city_matches[endpoint_cols].add_prefix("dep_").rename(columns={"dep_iata": "dep_iata"})
arr_endpoints = airport_city_matches[endpoint_cols].add_prefix("arr_").rename(columns={"arr_iata": "arr_iata"})

flights_enriched = (
    flights_df.dropna(subset=["dep_iata", "arr_iata"])
    .merge(dep_endpoints, on="dep_iata", how="left")
    .merge(arr_endpoints, on="arr_iata", how="left")
)

arr_tourism = tourism[[
    "country_iso3", "tourism_arrivals", "tourism_arrivals_year",
    "tourism_receipts_usd", "tourism_receipts_usd_year", "income_level",
]]
flights_enriched = flights_enriched.merge(arr_tourism, on="arr_country_iso3", how="left")
```

ILLUSTRATIVE, NOT EXHAUSTIVE

- Matched airports to its nearest WUP Top-100 city within a 250 km Haversine distance threshold, producing **989 valid airport-to-city links** across countries
- Attached **World Bank tourism and GDP** to each route end using ISO3 country codes
- Filtered to intercity routes only, yielding **217 intercity flights** that aggregated into **115 unique route records** with full demand context



Feature Engineering

ILLUSTRATIVE, NOT EXHAUSTIVE

```

routes["route"] = routes["dep_iata"] + "-" + routes["arr_iata"]
routes["city_pair"] = routes["dep_wup_city"] + " to " + routes["arr_wup_city"]
routes["continent_pair"] = routes.apply(lambda row: "-".join(sorted([row["dep_airport_continent"], row["arr_airport_continent"]])), axis=1)
routes["is_domestic"] = routes["dep_country_code"] == routes["arr_country_code"]
routes["route_distance_km"] = routes.apply(lambda row: haversine_km(row["dep_airport_latitude"], row["dep_airport_longitude"], row["arr_airport_latitude"], row["arr_airport_longitude"]), axis=1)
routes["population_demand_score"] = routes["dep_city_population"] * routes["arr_city_population"]
routes["population_underserved_score"] = routes["population_demand_score"] / routes["flight_count"].replace(0, np.nan)
routes["tourism_adjusted_demand"] = routes["population_demand_score"] * routes["arr_tourism_arrivals"]
routes["tourism_underserved_score"] = routes["tourism_adjusted_demand"] / routes["flight_count"].replace(0, np.nan)
routes["dep_log_gdp"] = np.log10(routes["dep_gdp_current_usd"])
routes["arr_log_gdp"] = np.log10(routes["arr_gdp_current_usd"])
routes["route_gdp_product"] = routes["dep_gdp_current_usd"] * routes["arr_gdp_current_usd"]
routes["route_gdp_mean"] = routes[["dep_gdp_current_usd", "arr_gdp_current_usd"]].mean(axis=1, skipna=False)
routes["route_log_gdp_mean"] = routes[["dep_log_gdp", "arr_log_gdp"]].mean(axis=1, skipna=False)
routes["economic_adjusted_demand"] = routes["population_demand_score"] * routes["arr_log_gdp"]
routes["economic_underserved_score"] = routes["economic_adjusted_demand"] / routes["flight_count"].replace(0, np.nan)
for raw_col, norm_col in [
    ("population_underserved_score", "population_underserved_norm"),
    ("tourism_underserved_score", "tourism_underserved_norm"),
    ("economic_underserved_score", "economic_underserved_norm"),
]:
    min_score = routes[raw_col].min(skipna=True)
    max_score = routes[raw_col].max(skipna=True)
    routes[norm_col] = (routes[raw_col] - min_score) / (max_score - min_score) if pd.notna(min_score) and pd.notna(max_score) and max_score > min_score else 0
routes["top_economic_gap"] = routes["economic_underserved_score"] >= routes["economic_underserved_score"].quantile(0.75)
routes = routes.sort_values("economic_underserved_score", ascending=False, na_position="last").reset_index(drop=True)

```

- **Constructed three demand scores:** population demand score , tourism-adjusted demand, and economic-adjusted demand, each divided by observed flight count to produce an underserved gap score
- **Log-transformed GDP values** to prevent the largest economies from dominating the ranking
- **Min-max normalized all three underserved scores to [0,1]** for comparability, derived a binary top_economic_gap label

